Лабораторная работа WT-3

Реализация MVC-архитектуры в консольном приложении C

Дисциплина: «Web-технологии»
Продолжительность: 4 академических часа
Максимальное количество студентов: 21
Среда разработки: Visual Studio или Visual Studio Code
Язык программирования: C#
Тип приложения: Console Application

---

1. Цель работы

Изучить принцип организации приложения по архитектурному шаблону **MVC (Model–View–Controller)** и реализовать его основные компоненты в консольном приложении на языке C#.

В ходе работы необходимо:

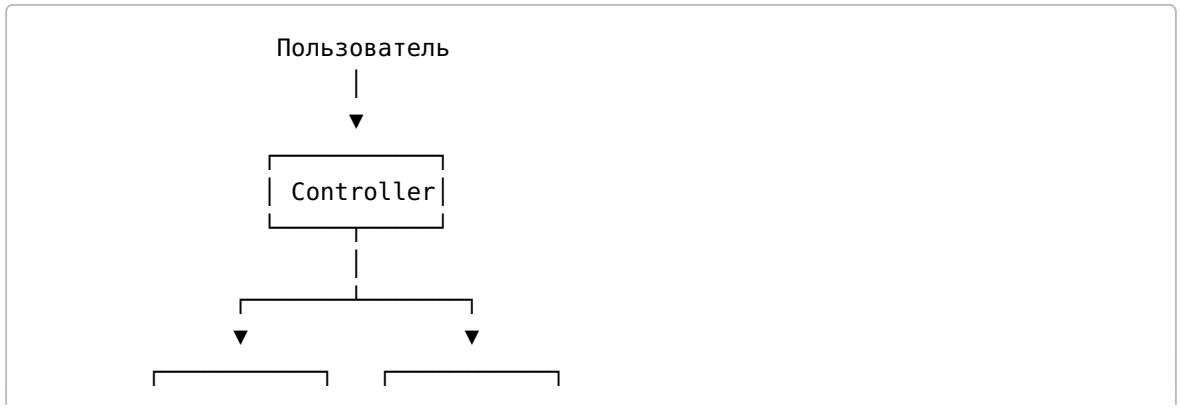
- создать консольное приложение на C#;
- создать модель предметной области;
- реализовать операции работы с объектами модели;
- создать представление для отображения данных;
- создать контроллер для обработки действий пользователя;
- организовать взаимодействие компонентов по принципу MVC;
- проверить работу приложения;
- опубликовать исходный код в репозитории Git.

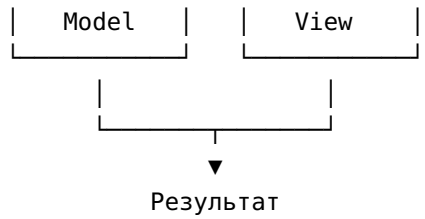
---

2. Теоретические сведения

2.1. Архитектурный шаблон MVC

MVC (Model–View–Controller) — архитектурный шаблон, предназначенный для разделения приложения на три взаимосвязанные части:





Model

Model содержит данные и логику предметной области.

Например, для интернет-магазина модель товара может содержать:

- идентификатор;
- название;
- категорию;
- цену;
- признак доступности.

View

View отвечает за представление информации пользователю.

В консольном приложении роль View выполняет код, который выводит информацию в консоль.

Controller

Controller принимает действия пользователя, обращается к Model и определяет, какие данные необходимо передать в View.

В настоящем веб-приложении Controller обрабатывает HTTP-запросы. В данной лабораторной работе аналогичная логика реализуется в консольном приложении.

2.2. MVC в веб-приложениях

В ASP.NET Core MVC архитектура имеет следующий вид:

```
HTTP-запрос
     │
     ▼
 Controller
     │
     ▼
   Model
     │
     ▼
   View
     │
```

```
     ▼
HTML-ответ
```

В данной лабораторной работе HTTP и HTML отсутствуют:

```
Пользователь
     |
     ▼
Controller
     |
     ▼
 Model
     |
     ▼
 View
     |
     ▼
Консоль
```

Таким образом, лабораторная работа позволяет изучить сам принцип разделения ответственности до перехода к ASP.NET Core MVC.

---

3. Постановка задачи

Необходимо разработать консольное приложение, моделирующее простую информационную систему.

Предметную область необходимо выбрать из списка:

№	Предметная область	Основная сущность
1	Интернет-магазин	Product
2	Библиотека	Book
3	Прокат автомобилей	Car
4	Банк	BankAccount
5	Медицинский центр	Patient
6	Университет	Student
7	Гостиница	Room
8	Авиакомпания	Flight
9	Доставка еды	Dish
10	Спортивный клуб	Member

№	Предметная область	Основная сущность
11	Управление IT-проектами	Task
12	Склад	Cargo
13	Музыкальный сервис	Track
14	Агентство недвижимости	Property
15	Турнир	Player

Конкретный вариант предметной области назначает преподаватель.

---

4. Требования к приложению

Приложение должно содержать три основных компонента:

```
Project
│
├── Models
│   └── Product.cs
│
├── Views
│   └── ProductView.cs
│
├── Controllers
│   └── ProductController.cs
│
└── Program.cs
```

Названия классов необходимо изменить в соответствии с выбранной предметной областью.

Например, для интернет-магазина:

```
Models/
    Product.cs

Views/
    ProductView.cs

Controllers/
    ProductController.cs
```

Для библиотеки:

```
Models/
    Book.cs

Views/
    BookView.cs

Controllers/
    BookController.cs
```

---

5. Выполнение работы

5.1. Создание проекта

Создать консольное приложение.

В Visual Studio выбрать:

Create a new project → Console App

В качестве имени проекта использовать:

```
wt-lab3-3mvc-ivanov
```

где `ivanov` заменить на свою фамилию.

При использовании Visual Studio Code выполнить:

```
dotnet new console -n wt-lab3-3mvc-ivanov
```

Перейти в каталог проекта:

```
cd wt-lab3-3mvc-ivanov
```

Проверить запуск:

```
dotnet run
```

---

6. Создание модели

6.1. Создание каталога Models

Создать каталог:

```
Models
```

Создать в нём класс выбранной предметной области.

Для примера используется интернет-магазин.

Создать файл:

```
Models/Product.cs
```

Содержимое:

```
namespace wt_lab3_3mvc_ivanov.Models;

public class Product
{
    public int Id { get; set; }

    public string Name { get; set; } = "";

    public string Category { get; set; } = "";

    public decimal Price { get; set; }

    public bool IsAvailable { get; set; }
}
```

Модель описывает **один объект предметной области**.

Например:

```
Id: 1
Name: Ноутбук
Category: Компьютеры
Price: 2500
IsAvailable: true
```

---

7. Создание представления

7.1. Создание каталога Views

Создать каталог:

```
Views
```

Создать файл:

```
Views/ProductView.cs
```

Добавить класс:

```
using wt_lab3_3mvc_ivanov.Models;

namespace wt_lab3_3mvc_ivanov.Views;

public class ProductView
{
    public void ShowProducts(List<Product> products)
    {
        Console.WriteLine("=== СПИСОК ТОВАРОВ ===");

        foreach (var product in products)
        {
            Console.WriteLine(
                $"{product.Id}. " +
                $"{product.Name} | " +
                $"{product.Category} | " +
                $"{product.Price} руб. | " +
                $"Доступен: {product.IsAvailable}"
            );
        }
    }

    public void ShowProduct(Product product)
    {
        Console.WriteLine("=== ТОВАР ===");

        Console.WriteLine($"ID: {product.Id}");
        Console.WriteLine($"Название: {product.Name}");
        Console.WriteLine($"Категория: {product.Category}");
        Console.WriteLine($"Цена: {product.Price} руб.");
        Console.WriteLine($"Доступен: {product.IsAvailable}");
    }
```

```
    public void ShowMessage(string message)
    {
        Console.WriteLine(message);
    }
}
```

View не должен самостоятельно искать товары или изменять данные.

Его задача — **отображать переданные данные**.

---

8. Создание контроллера

8.1. Создание каталога Controllers

Создать каталог:

```
Controllers
```

Создать файл:

```
Controllers/ProductController.cs
```

Добавить:

```
using wt_lab3_3mvc_ivanov.Models;
using wt_lab3_3mvc_ivanov.Views;

namespace wt_lab3_3mvc_ivanov.Controllers;

public class ProductController
{
    private readonly List<Product> products;
    private readonly ProductView view;

    public ProductController()
    {
        products = new List<Product>
        {
            new Product
            {
                Id = 1,
                Name = "Ноутбук",
                Category = "Компьютеры",
                Price = 2500,
                IsAvailable = true
```

```
            },
            new Product
            {
                Id = 2,
                Name = "Мышь",
                Category = "Аксессуары",
                Price = 80,
                IsAvailable = true
            },
            new Product
            {
                Id = 3,
                Name = "Клавиатура",
                Category = "Аксессуары",
                Price = 150,
                IsAvailable = false
            }
        };

        view = new ProductView();
    }

    public void Index()
    {
        view.ShowProducts(products);
    }

    public void Show(int id)
    {
        var product = products.FirstOrDefault(
            p => p.Id == id
        );

        if (product == null)
        {
            view.ShowMessage("Товар не найден.");
            return;
        }

        view.ShowProduct(product);
    }

    public void Add(Product product)
    {
        products.Add(product);

        view.ShowMessage(
            "Товар успешно добавлен."
        );
    }
```

```
    public void Delete(int id)
    {
        var product = products.FirstOrDefault(
            p => p.Id == id
        );

        if (product == null)
        {
            view.ShowMessage("Товар не найден.");
            return;
        }

        products.Remove(product);

        view.ShowMessage(
            "Товар успешно удалён."
        );
    }
}
```

Контроллер выполняет несколько действий:

```
Index()  → получить список объектов
Show()   → получить один объект
Add()    → добавить объект
Delete() → удалить объект
```

---

9. Запуск MVC из Program.cs

Файл `Program.cs` должен создать контроллер и вызвать его методы.

Использовать:

```
using wt_lab3_3mvc_ivanov.Controllers;
using wt_lab3_3mvc_ivanov.Models;

var controller = new ProductController();

controller.Index();

Console.WriteLine();

controller.Show(2);

Console.WriteLine();
```

```
controller.Add(
    new Product
    {
        Id = 4,
        Name = "Монитор",
        Category = "Компьютеры",
        Price = 900,
        IsAvailable = true
    }
);

Console.WriteLine();

controller.Index();

Console.WriteLine();

controller.Delete(2);

Console.WriteLine();

controller.Index();
```

Запустить приложение:

```
dotnet run
```

---

10. Проверка взаимодействия компонентов

При выполнении программы должна прослеживаться следующая последовательность:

```
Program
   |
   ▼
Controller
   |
   ▼
Model
   |
   ▼
Controller
   |
   ▼
View
   |
```

```
▼
Console
```

Например, при выполнении:

```
controller.Show(2);
```

контроллер:

1. получает идентификатор товара;
2. выполняет поиск объекта в Model;
3. передаёт найденный объект в View;
4. View выводит информацию.

Таким образом, `Program.cs` не должен самостоятельно выполнять поиск и форматирование товара.

---

11. Добавление пользовательского меню

Для закрепления принципа MVC реализовать консольное меню.

Создать в `ProductController` метод:

```
public void Run()
{
    while (true)
    {
        Console.WriteLine();
        Console.WriteLine("=== МЕНЮ ===");
        Console.WriteLine("1. Показать товары");
        Console.WriteLine("2. Найти товар");
        Console.WriteLine("3. Добавить товар");
        Console.WriteLine("4. Удалить товар");
        Console.WriteLine("0. Выход");

        Console.Write("Выберите действие: ");

        var command = Console.ReadLine();

        switch (command)
        {
            case "1":
                Index();
                break;

            case "2":
                ShowProductByUserInput();
```

```
                break;

            case "3":
                AddProductByUserInput();
                break;

            case "4":
                DeleteProductByUserInput();
                break;

            case "0":
                return;

            default:
                view.ShowMessage(
                    "Неизвестная команда."
                );
                break;
        }
    }
}
```

Дополнить контроллер методами обработки пользовательского ввода:

```
private void ShowProductByUserInput()
{
    Console.Write("Введите ID товара: ");

    if (!int.TryParse(
        Console.ReadLine(),
        out int id))
    {
        view.ShowMessage("Некорректный ID.");
        return;
    }

    Show(id);
}
```

Для добавления и удаления реализовать аналогичную обработку.

После этого изменить `Program.cs`:

```
using wt_lab3_3mvc_ivanov.Controllers;

var controller = new ProductController();
```

```
controller.Run();
```

---

12. Адаптация к выбранной предметной области

После выполнения примера необходимо заменить предметную область на назначенный вариант.

Например, для библиотеки модель может выглядеть следующим образом:

```
namespace wt_lab3_3mvc_ivanov.Models;

public class Book
{
    public int Id { get; set; }

    public string Title { get; set; } = "";

    public string Author { get; set; } = "";

    public int Year { get; set; }

    public bool IsAvailable { get; set; }
}
```

Для выбранной предметной области необходимо самостоятельно определить:

- класс модели;
- свойства модели;
- начальные объекты;
- операции контроллера;
- отображение данных в View.

---

13. Требования к архитектуре

В приложении необходимо обеспечить разделение ответственности.

Model

Должна содержать:

- свойства объекта;
- структуру данных предметной области.

View

Должна содержать:

- вывод информации;
- форматирование информации;
- сообщения пользователю.

Controller

Должен содержать:

- получение команд;
- обработку действий;
- поиск объектов;
- добавление объектов;
- удаление объектов;
- взаимодействие с Model и View.

Не следует размещать всю программу в `Program.cs`.

Необходимо обеспечить структуру:

```
Program
   ↓
Controller
   ↓
Model
   ↓
View
```

При этом `View` не должна самостоятельно изменять коллекцию объектов.

---

14. Дополнительное задание

Реализовать одну дополнительную операцию в соответствии с выбранной предметной областью.

Возможные варианты:

- поиск по названию;
- фильтрация по категории;
- сортировка;
- изменение объекта;
- поиск доступных объектов;
- вычисление общей стоимости;
- подсчёт количества объектов.

Например, для товаров:

```
public void Search(string text)
{
    var result = products
        .Where(p =>
            p.Name.Contains(
                text,
                StringComparison.OrdinalIgnoreCase
            ))
        .ToList();

    view.ShowProducts(result);
}
```

Добавить соответствующий пункт в консольное меню.

---

15. Проверка работы

Проверить следующие сценарии:

1. запустить приложение;
2. вывести список объектов;
3. вывести один объект по идентификатору;
4. выполнить поиск несуществующего объекта;
5. добавить новый объект;
6. повторно вывести список;
7. удалить объект;
8. повторно вывести список;
9. выполнить дополнительную операцию;
10. проверить обработку некорректного пользовательского ввода.

---

16. Работа с Git

Инициализировать Git-репозиторий:

```
git init
```

Добавить файлы:

```
git add .
```

Создать первый коммит:

```
git commit -m "Создана MVC-структура приложения"
```

После завершения работы создать финальный коммит:

```
git add .
git commit -m "Завершена лабораторная работа WT-3"
```

Создать репозиторий на GitHub и опубликовать проект.

Название репозитория:

```
wt-lab3-3mvc-ivanov
```

---

17. Отчёт

Создать отчёт в формате PDF.

В отчёт включить:

1. титульную страницу;
2. номер и название лабораторной работы;
3. вариант предметной области;
4. структуру проекта;
5. код класса модели;
6. код View;
7. код Controller;
8. код `Program.cs`;
9. скриншот структуры проекта;
10. скриншоты работы приложения;
11. пример добавления объекта;
12. пример удаления объекта;
13. пример дополнительной операции;
14. ссылку на GitHub-репозиторий.

---

18. Контрольные вопросы

1. Что такое архитектурный шаблон MVC?
2. Для чего используется Model?
3. Для чего используется View?
4. Для чего используется Controller?
5. Чем Model отличается от View?
6. Почему не следует размещать всю логику приложения в `Program.cs`?
7. Как Controller взаимодействует с Model?

8. Как Controller взаимодействует с View?
9. Какую роль выполняет `Product` в приложении?
10. Почему данные модели не должны форматироваться непосредственно в Controller?
11. Для чего используется `List<T>`?
12. Для чего используется `FirstOrDefault()`?
13. Для чего используется `Where()`?
14. Что произойдёт, если объект с указанным ID отсутствует?
15. Почему обработку ошибок необходимо учитывать при работе с пользовательским вводом?
16. Что такое разделение ответственности?
17. Чем MVC в консольном приложении отличается от MVC в веб-приложении?
18. Какой компонент ASP.NET Core MVC отвечает за обработку HTTP-запроса?
19. Какую роль выполняет View в ASP.NET Core MVC?
20. Какие преимущества даёт разделение приложения на Model, View и Controller?

---

19. Критерии оценки

Максимальная оценка — **21 балл**.

Этап	1 балл	2 балла	3 балла
Создание проекта	проект создан	проект запускается	проект корректно структурирован
Model	создан класс	добавлены свойства	модель соответствует предметной области
View	создан View	реализован вывод	реализованы разные варианты отображения
Controller	создан Controller	реализованы операции	обеспечено взаимодействие Model и View
Меню и обработка	создано меню	реализованы команды	обработаны ошибки ввода
Дополнительная операция	операция отсутствует/не работает	операция реализована	операция корректно интегрирована в MVC
Git и отчёт	выполнено частично	проект опубликован	отчёт полностью оформлен

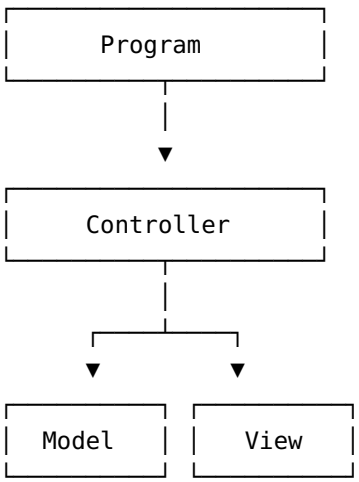
---

20. Результат работы

В результате выполнения лабораторной работы необходимо получить консольное приложение следующей структуры:

```
wt-lab3-3mvc-ivanov/
│
├── Models/
│   └── Product.cs
│
├── Views/
│   └── ProductView.cs
│
├── Controllers/
│   └── ProductController.cs
│
├── Program.cs
├── wt-lab3-3mvc-ivanov.csproj
└── ...
```

Архитектура приложения:



В следующей лабораторной работе эта архитектура переносится из консольного приложения в **ASP.NET Core MVC**, где Controller будет обрабатывать HTTP-запросы, Model — представлять данные приложения, а View — формировать HTML-страницы.